

LAPORAN UAS MINI PROJECT

COMMUNICATION PROTOCOLS DATA SCIENCE PROFESIONAL

Monitoring Tracking Order Sistem – REST API



Disusun oleh:
Galih Agung Nurfadhilah (25110500026)

**UAS BLOK B SEMESTER II
2026**

A. Problem Statement & Tujuan Mini Project

Latar Belakang

Dalam proses jual-beli online, baik pembeli maupun penjual membutuhkan cara untuk memantau status pesanan secara real-time sepanjang siklus hidup order — mulai dari pesanan dibuat, diproses, dikirim, hingga pembayaran diselesaikan. Tanpa mekanisme komunikasi yang jelas antara sisi pembeli dan penjual, informasi status transaksi rawan tertinggal atau tidak sinkron. Kebutuhan inilah yang mendasari pemilihan studi kasus pada mini project ini.

Use Case yang Dipilih

Studi kasus yang digunakan adalah pembelian laptop **ASUS TUF A15 Ryzen 7** seharga **Rp14.500.000** oleh seorang pembeli bernama **Galih**. Skenario ini merepresentasikan alur order dan transaksi yang umum terjadi pada platform e-commerce: pembeli membuat order, penjual memperbarui status order (diproses ☐ dikirim), sistem membuat transaksi pembayaran, dan status pembayaran diperbarui hingga selesai.

Alasan Pemilihan Use Case

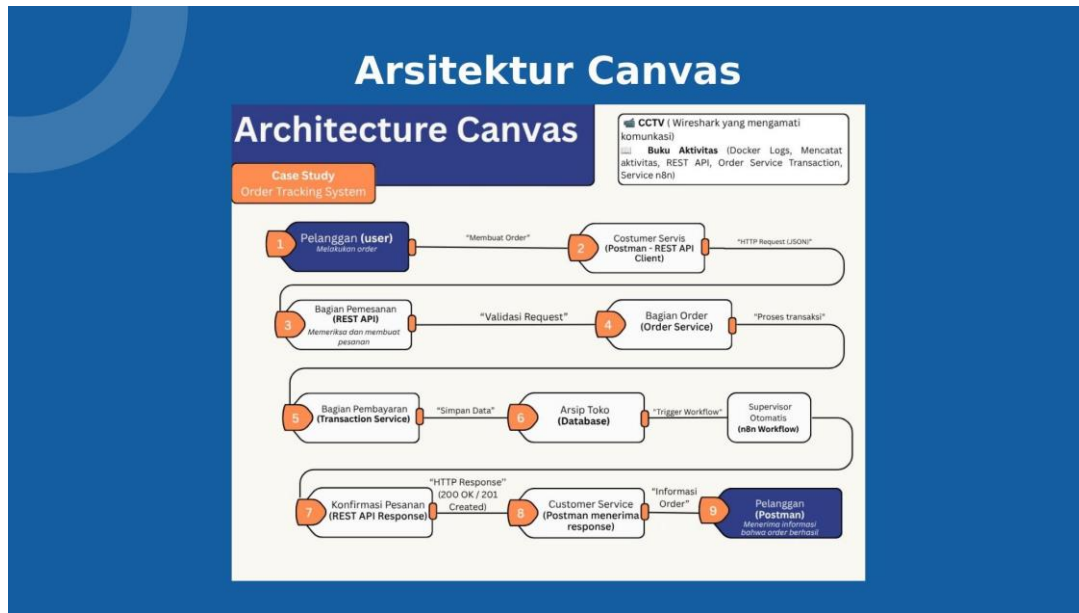
- Order dan transaksi merupakan dua resource yang saling berelasi, sehingga cocok untuk mendemonstrasikan operasi CRUD (Create, Read, Update, Patch, Delete) secara lengkap pada dua entitas sekaligus.
- Alur status order (created ☐ processing ☐ shipped) dan transaksi (pending ☐ paid ☐ completed) merepresentasikan siklus hidup nyata yang membutuhkan komunikasi API berulang antara pembeli dan penjual.
- Skenario ini memungkinkan pengujian success scenario maupun failure/error scenario, serta memberi ruang untuk menambahkan reliability layer tambahan melalui webhook n8n.

Tujuan Mini Project

Tujuan dari project ini adalah membuktikan kemampuan penulis dalam **merancang, menjalankan, dan menguji komunikasi API** untuk siklus hidup sebuah order — meliputi perancangan endpoint REST, eksekusi request menggunakan Postman, verifikasi lalu lintas jaringan menggunakan Wireshark, serta implementasi reliability check tambahan menggunakan workflow n8n.

B. Architecture Canvas & Data Flow Diagram

Architecture canvas berikut menggambarkan komponen-komponen yang terlibat dalam sistem monitoring order — mulai dari client (Postman sebagai simulasi buyer/seller), REST API server yang menangani resource **orders** dan **transactions**, hingga reliability layer berbasis workflow n8n.



Gambar 1. Architecture Canvas sistem Monitoring Tracking Order

Alur Data (Data Flow)

- **Buyer** mengirim **POST /orders** untuk membuat order baru (contoh: pembelian ASUS TUF A15 Ryzen 7).
- **Seller** memantau order melalui **GET /orders/{id}**, kemudian memperbarui status menjadi *processing* via **PUT /orders/{id}**.
- **Buyer** memverifikasi perubahan status melalui **GET /orders/{id}**, lalu seller menandai order sebagai *shipped* via **PATCH /orders/{id}**.
- Sistem membuat **transaksi pembayaran** terkait order melalui **POST /transactions**, kemudian status pembayaran diperbarui melalui **PUT** dan **PATCH /transactions/{id}** hingga berstatus *completed*.
- Setiap perubahan diverifikasi kembali menggunakan **GET**, dan salah satu transaksi lama dihapus melalui **DELETE /transactions/{id}** untuk menguji operasi delete.
- Sebagai reliability layer tambahan, request order dikirim juga ke **webhook n8n** untuk disimulasikan skenario sukses (200 OK) dan gagal (404 Not Found).

C. Protocol Selection

Protokol Utama: REST API (HTTP)

Project ini menggunakan **REST API berbasis HTTP** sebagai protokol komunikasi utama antara client (Postman) dan server pada **http://localhost:8088**. REST dipilih karena beberapa pertimbangan berikut:

- Resource **orders** dan **transactions** bersifat terstruktur dan berorientasi CRUD, sehingga selaras dengan prinsip REST yang memetakan operasi (Create, Read, Update, Patch, Delete) ke method HTTP standar (POST, GET, PUT, PATCH, DELETE).
- Model komunikasi request–response yang sinkron pada REST sesuai dengan kebutuhan use case, di mana buyer dan seller perlu mendapatkan konfirmasi langsung atas setiap aksi (misalnya status code dan response body) setelah request dikirim.
- REST API mudah diuji secara manual maupun otomatis menggunakan tools seperti Postman, serta lalu lintasnya dapat diverifikasi menggunakan packet analyzer seperti Wireshark, sehingga cocok untuk kebutuhan evidence dan observability pada project ini.
- REST tidak memerlukan koneksi persisten seperti WebSocket, sehingga lebih sederhana untuk disimulasikan dalam lingkungan lokal/mock service pada mini project ini.

Protokol Tambahan: Webhook n8n

Sebagai lapisan tambahan (reliability layer), project ini juga memanfaatkan **webhook n8n**. Alasan pemilihan n8n webhook:

- n8n memungkinkan pembuatan workflow otomatis yang dapat menerima request HTTP (webhook) dan menjalankan proses validasi/reliability check tanpa perlu membangun service tambahan dari nol.
- Webhook n8n digunakan untuk mensimulasikan skenario sukses (response 200 OK) dan skenario gagal (response 404 Not Found) pada request Post Order, sehingga dapat menunjukkan bagaimana sistem menangani failure scenario di luar REST API utama.
- Execution history pada n8n memberikan observability tambahan berupa log eksekusi yang dapat dijadikan bukti reliability di luar log Postman dan Wireshark.

Protokol lain seperti WebSocket, MQTT, dan gRPC tidak digunakan pada project ini karena use case monitoring order tidak memerlukan koneksi dua-arah yang persisten (WebSocket/MQTT) maupun komunikasi antar-service berperforma tinggi dengan skema biner (gRPC) — kebutuhan request/response sinkron pada REST API sudah cukup untuk mendemonstrasikan siklus hidup order dan transaksi.

D. Endpoint / Action Design

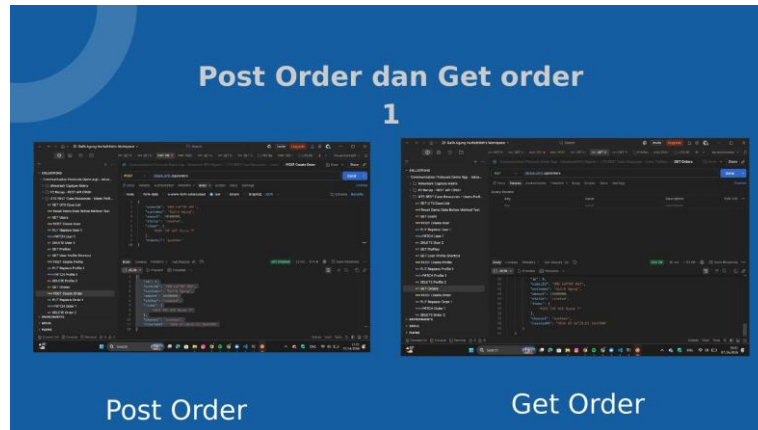
Base URL: <http://localhost:8088> | Header umum: Content-Type: application/json

No	Method	Path	Request Body (ringkas)	Response Body (ringkas)	Expected Status
1	POST	/orders	id, buyer, item, price, status: "created"	Order tersimpan dengan id & status "created"	201 Created
2	GET	/orders/{id}	-	Detail order (buyer menerima info order)	200 OK
3	PUT	/orders/{id}	status: "processing"	Order dengan status ter-update "processing"	200 OK
4	GET	/orders/{id}	-	Order status "processing"	200 OK
5	PATCH	/orders/{id}	status: "shipped"	Order dengan status ter-update "shipped"	200 OK
6	GET	/orders/{id}	-	Order status "shipped"	200 OK
7	POST	/transactions	order_id, amount, status: "pending" (id: 3)	Transaksi baru tersimpan (id: 3)	201 Created
8	GET	/transactions/{id}	-	Detail status pembayaran	200 OK
9	PUT	/transactions/{id}	status: "paid"	Transaksi ter-update status "paid"	200 OK
10	GET	/transactions/{id}	-	Status pembayaran terkini	200 OK
11	PATCH	/transactions/{id}	status: "completed"	Transaksi status "completed"	200 OK
12	GET	/transactions/{id}	-	Status pembayaran "completed"	200 OK
13	DELETE	/transactions/{id=2}	-	Transaksi id:2 dihapus	200 OK / 204 No Content
14	GET	/transactions/{id=3}	-	Transaksi id:3 tetap ada (tidak terhapus)	200 OK

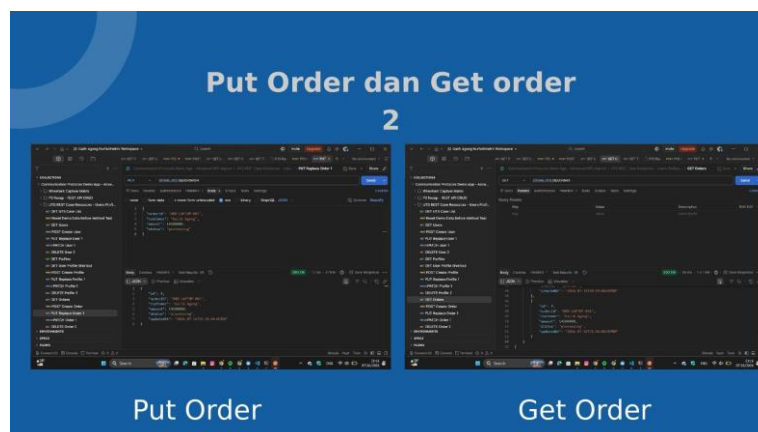
Tabel 1. Rancangan 14 endpoint/action pada resource **orders** dan **transactions**, mencakup seluruh siklus hidup order (created ☐ processing ☐ shipped) dan transaksi (pending ☐ paid ☐ completed), termasuk operasi delete pada transaksi id:2 yang tidak mempengaruhi transaksi id:3.

E. Evidence — Success Scenario (Order)

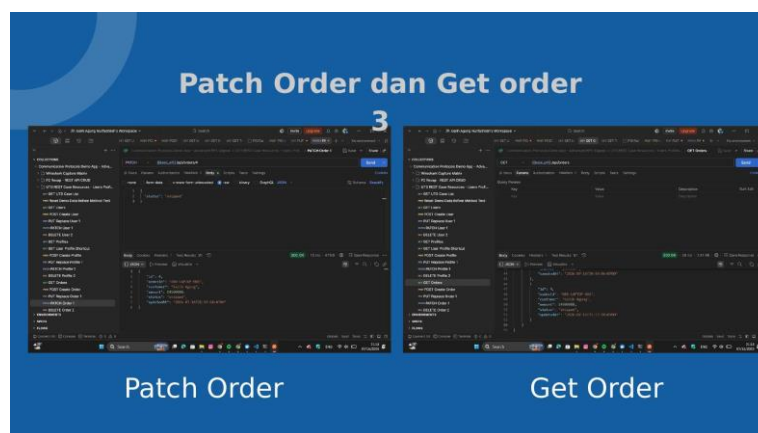
Berikut bukti eksekusi request pada resource **orders** menggunakan Postman, mencakup seluruh perubahan status order dari *created* hingga *shipped*.



Gambar 2. POST Order (create) dan GET Order 1 — status "created"



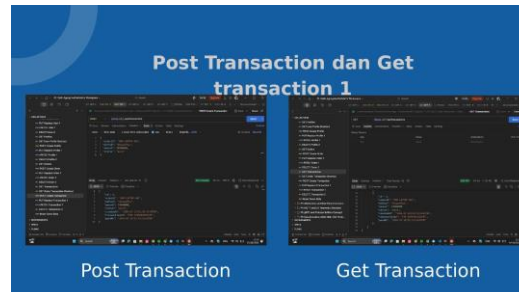
Gambar 3. PUT Order dan GET Order 2 — status ter-update menjadi "processing"



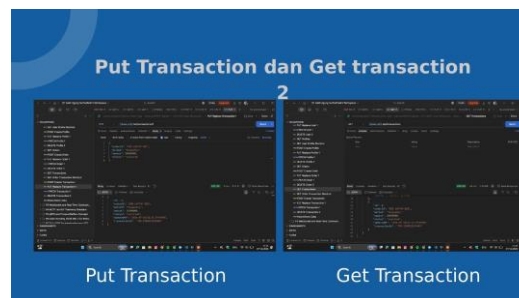
Gambar 4. PATCH Order dan GET Order 3 — status ter-update menjadi "shipped"

F. Evidence — Success & Failure Scenario (Transaction & n8n)

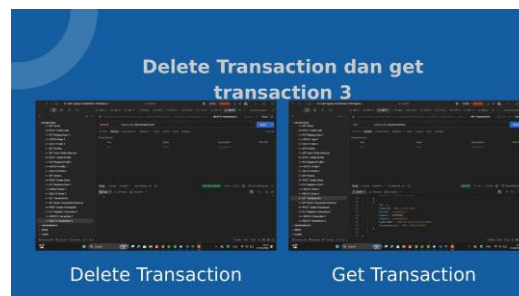
Evidence berikut menunjukkan siklus transaksi pembayaran (pending $\square$ paid $\square$ completed), operasi delete yang tidak mempengaruhi transaksi lain, serta skenario sukses dan gagal pada webhook n8n.



Gambar 5. POST Transaction (create id:3) dan GET Transaction 1

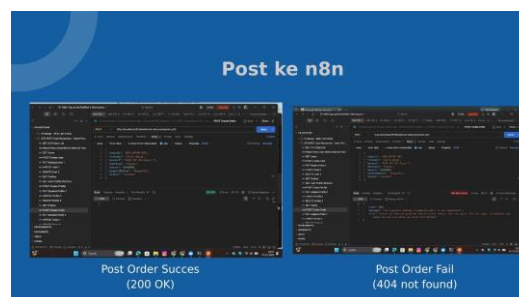


Gambar 6. PUT Transaction dan GET Transaction 2 — status "paid"



Gambar 7. DELETE Transaction (id:2) dan GET Transaction 3 — id:3 tidak terhapus

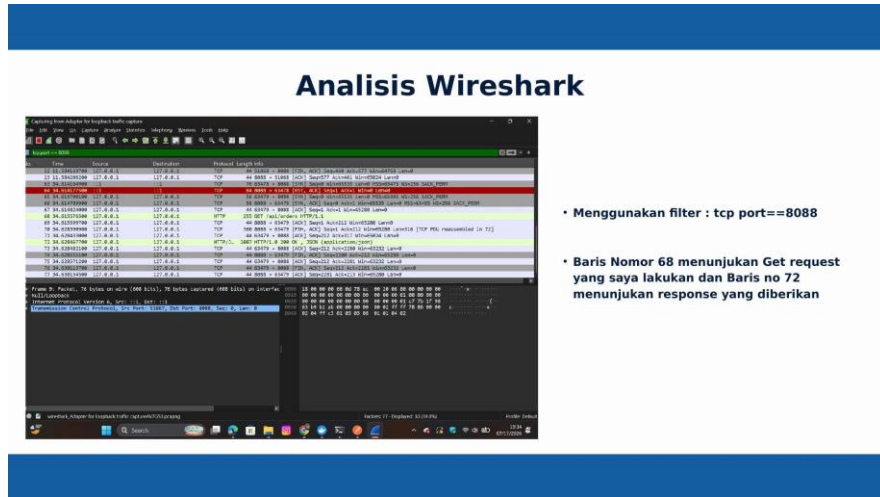
Failure Scenario — Webhook n8n



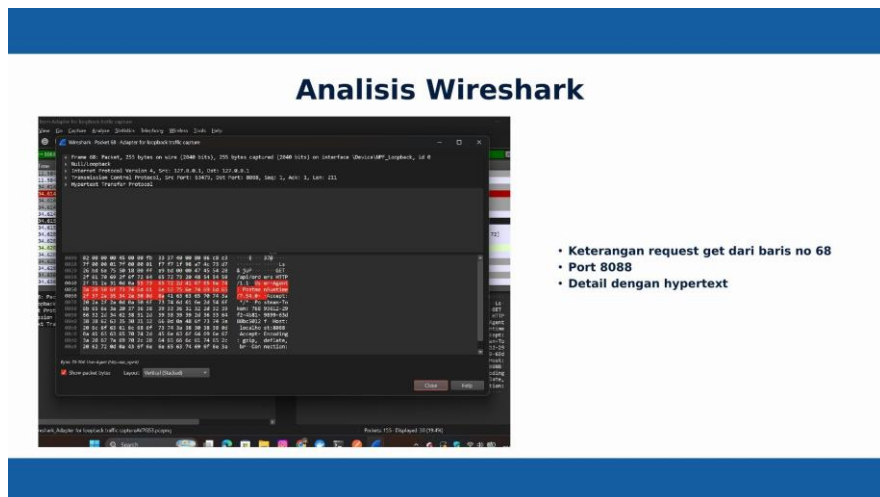
Gambar 8. Post Order ke n8n — skenario sukses (200 OK) dan skenario gagal (404 Not Found) sebagai bukti failure/error scenario pada reliability layer.

G. Observability & Traffic Evidence

Untuk memverifikasi lalu lintas komunikasi API secara langsung pada level jaringan, dilakukan capture menggunakan **Wireshark** dengan filter tcp port==8088, sesuai dengan port server REST API yang digunakan (localhost:8088).



Gambar 9. Capture Wireshark dengan filter tcp port==8088 — baris no. 68 menunjukan GET request yang dikirim, dan baris no. 72 menunjukan response yang diterima dari server.



Gambar 10. Detail request GET pada baris no. 68 — menunjukan port 8088 dan detail hypertext (HTTP) dari request yang dikirim.

Observability Tambahan (n8n Execution History)

Selain Wireshark, workflow n8n yang dijalankan sebagai reliability layer juga menyediakan **execution history** yang mencatat setiap eksekusi webhook — termasuk waktu eksekusi dan hasil (success/fail) — sehingga dapat digunakan sebagai bukti observability tambahan di luar log Postman dan packet capture Wireshark.

H. Testing Result & Reliability Analysis

Ringkasan Hasil Pengujian

Aspek	Hasil
Total request diuji	14 request (7 pada resource orders , 7 pada resource transactions)
Method HTTP tercakup	POST, GET, PUT, PATCH, DELETE
Success scenario	Seluruh request pada alur utama berhasil dieksekusi dengan status code sesuai ekspektasi (2xx)
Failure/error scenario	Diuji melalui webhook n8n — Post Order Fail merespons 404 Not Found
Reliability check	Webhook n8n digunakan sebagai reliability layer tambahan di luar REST API utama
Traffic verification	Divalidasi menggunakan Wireshark dengan filter tcp port==8088
Konsistensi data	DELETE pada transaksi id:2 terbukti tidak mempengaruhi transaksi id:3 (data lain tetap konsisten)

Analisis Reliability

- Seluruh 14 request pada alur CRUD utama (order & transaksi) berhasil dijalankan tanpa error, menunjukkan bahwa desain endpoint REST API sudah konsisten dengan siklus hidup order yang direncanakan.
- Penambahan reliability layer berupa webhook n8n memberikan lapisan pengujian tambahan di luar REST API utama, memungkinkan simulasi kondisi sukses (200 OK) maupun gagal (404 Not Found) secara terpisah.
- Verifikasi menggunakan Wireshark membuktikan bahwa komunikasi benar-benar terjadi pada level jaringan (port 8088), bukan hanya pada level aplikasi (Postman), sehingga meningkatkan kepercayaan terhadap hasil pengujian.
- Operasi DELETE pada transaksi id:2 diverifikasi tidak menghapus atau mengubah transaksi id:3, membuktikan bahwa operasi bersifat idempotent/terisolasi terhadap resource lain.

I. Kesimpulan, Limitation, Improvement & Reflection

Kesimpulan

Project Order Monitoring Laptop ini berhasil mendemonstrasikan alur CRUD lengkap pada dua resource, yaitu **order** dan **transaksi**, dengan total **14 request** yang mencakup skenario sukses maupun gagal, serta reliability check melalui webhook n8n. Hasil pengujian menunjukkan bahwa seluruh endpoint bekerja sesuai rancangan, dan komunikasi API dapat diverifikasi secara konsisten baik melalui Postman maupun packet capture Wireshark.

Limitation (Keterbatasan)

- Endpoint belum dilengkapi mekanisme autentikasi/otorisasi (misalnya token/API key), sehingga seluruh request masih dapat diakses tanpa validasi identitas pengirim.
- Data order dan transaksi disimulasikan pada mock service/local server sehingga belum menggunakan database persisten yang mendukung skala produksi.
- Pengujian reliability masih dilakukan secara manual melalui Postman dan n8n, belum mencakup simulasi kondisi jaringan yang lebih kompleks seperti timeout, retry otomatis, atau request bersamaan (concurrent).

Improvement (Rencana Pengembangan)

- Menambahkan autentikasi token pada setiap endpoint untuk memastikan hanya pihak yang berwenang (buyer/seller) yang dapat mengakses atau mengubah data order dan transaksi.
- Menambahkan notifikasi otomatis (misalnya melalui email atau push notification) setiap kali status order berubah, sehingga buyer dan seller tidak perlu melakukan polling secara manual.
- Melengkapi pengujian reliability dengan skenario yang lebih menyeluruh, seperti simulasi timeout, mekanisme retry otomatis, dan pengujian pada kondisi request bersamaan (concurrent load).

Reflection Pembelajaran

Melalui mini project ini, penulis memperoleh pemahaman praktis mengenai perancangan endpoint REST API yang selaras dengan siklus hidup bisnis nyata (order & transaksi), pentingnya pengujian success dan failure scenario secara terpisah, serta manfaat menggabungkan beberapa alat observability (Postman, Wireshark, dan execution history n8n) untuk membuktikan bahwa komunikasi API benar-benar berjalan sesuai rancangan — bukan hanya secara logis di level aplikasi, tetapi juga secara nyata di level jaringan.

Repository GitHub	https://github.com/gannnss/CommunicationProtocol-UAS
Video YouTube	https://youtu.be/ldwqlH60gZk